

Make model output reliable.

Ask for a shape. Check the shape. Plan for failure.

Messy reviews



Structured data



Validated result

TODAY'S
OUTCOME

Build three layers of protection.

1

Validate

Check every field and allowed value.

2

Retry

Try again within a fixed limit.

3

Fallback

Return an honest needs-review result.

**REAL
WORK
PROBLEM**

Software cannot reliably sort a changing paragraph.

One answer

A person can read a friendly paragraph.

Hundreds of reviews

A team needs consistent fields it can count, filter, and route.

MENTAL
MODEL

Conversation versus a filled-in form.

Chatty

"The customer seems
unhappy because the
charger stopped..."

Shape can move.

Structured

```
{"sentiment":"negative",  
  "topic":"charging",  
  "summary":"Stopped  
charging quickly"}
```

SYSTEM
INSTRUCTION

Give the model a job description.



Remove the important guesswork.

Role

Who?

Task

Do what?

Rules

Boundaries?

Unsure

When
unclear?

Format

What
shape?

STRUCTURED
OUTPUT

JSON is a filled-in form for software.

One record: { }

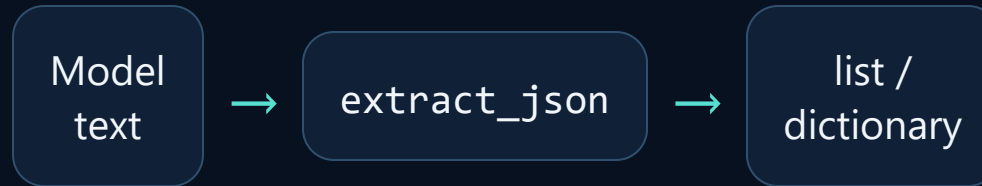
```
{"sentiment": "positive",  
  "topic": "delivery",  
  "summary": "Arrived  
early"}
```

List: []

```
[  
  
  {"sentiment": "positive",  
    ...},  
  
  {"sentiment": "negative",  
    ...}  
]
```

PARSING

JSON text becomes Python data.



Parsing can fail.
Parsed data can still
have the wrong
fields.

TWO
DIFFERENT
CHECKS

Valid JSON is not automatically valid business data.

Parse check

Can the text become data?

Schema check

Are required fields and allowed values present?

Meaning still needs evaluation after shape checks pass.

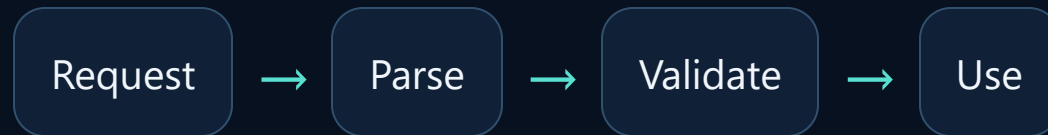
SCHEMA

Describe exactly what we accept.

```
class Review(BaseModel):  
    sentiment: Literal[  
        "positive", "negative",  
        "neutral", "unclear"  
    ]  
    topic: str  
    summary: str
```

RELIABILITY
PIPELINE

Use data only after it passes.



Failure:
bounded retry
→ flagged
fallback.

FINISHED
RESULT

Valid rows plus rejected bad data.

Happy path

positive |
delivery |
Arrived early
negative |
reliability |
Stopped
charging

Forced failure

REJECTED
ecstatic
REJECTED
missing
summary

Illustrative shape
until live output is
observed.

LAB
2.1

Request and parse a JSON array.

```
reply = chat(messages, system=SYSTEM)
results = extract_json(reply.text)

for item in results:
    print(item["sentiment"], item["topic"],
          item["summary"])
```

LIVE
DEMO

Run the structured sorter.

```
python phase-1-foundations/day-02/labs/  
lab-2.1-review-  
sorter/solution/review_sorter.py
```

Success

Five rows, three
required fields per row.

Provider failure

Report the error.
Continue to local
validation proof.

LAB
2.2

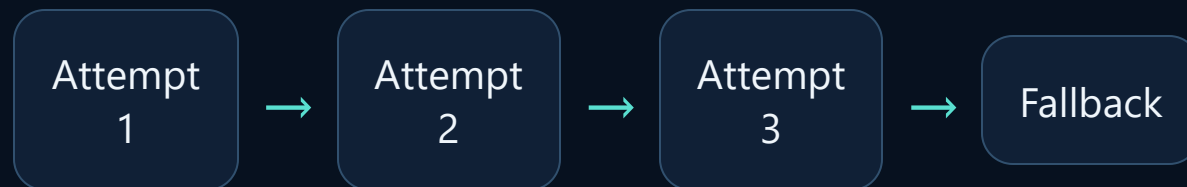
Validate before returning.

```
raw =  
extract_json(reply.text)  
return Review(**raw)
```

The function returns only after the schema accepts the data.

BOUNDED
RETRY

Try again, but know when to stop.



**SAFE
FALLBACK**

Do not disguise failure as success.

```
Review(  
  sentiment="unclear",  
  topic="needs-review",  
  summary="Could not classify  
automatically"  
)
```

**DETERMINISTIC
FAILURE**

Force known bad data.

Bad value

```
{"sentiment": "ecstatic",  
  "topic": "x",  
  "summary": "y"}
```

Missing field

```
{"sentiment": "positive",  
  "topic": "x"}
```

**PRODUCTION
BOUNDARY**

Shape checks are necessary, not sufficient.

Validity

Fields and allowed
values.

Meaning

Is the classification
actually right?

Operations

Timeout, backoff,
queue, owner,
alert.

COMPLETION
GATE

Day 2 is complete when failure is handled.

- ✓ Structured sorter produces required fields.
- ✓ Known malformed records are rejected.
- ✓ Retry count has a hard limit.
- ✓ Fallback is clearly marked for human review.
- ✓ Evidence notes state results and limits.

Next: just-enough
Python for
maintainable
workflows.